



AI Deployment

Déploiement de solutions d'IA

MSc 2025-2026

IA appliquée au renseignement OSINT

Partie 1

Le point de départ

On dispose d'un corpus d'articles issus de la rubrique « **Guerre** » de la plateforme tass.com, en anglais, au format **JSON**.

EUGENIASCHOOL

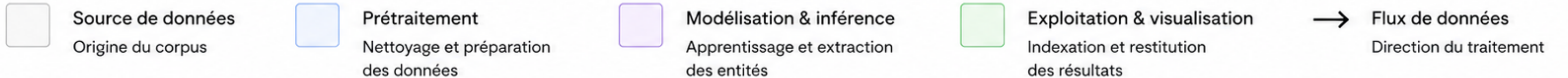
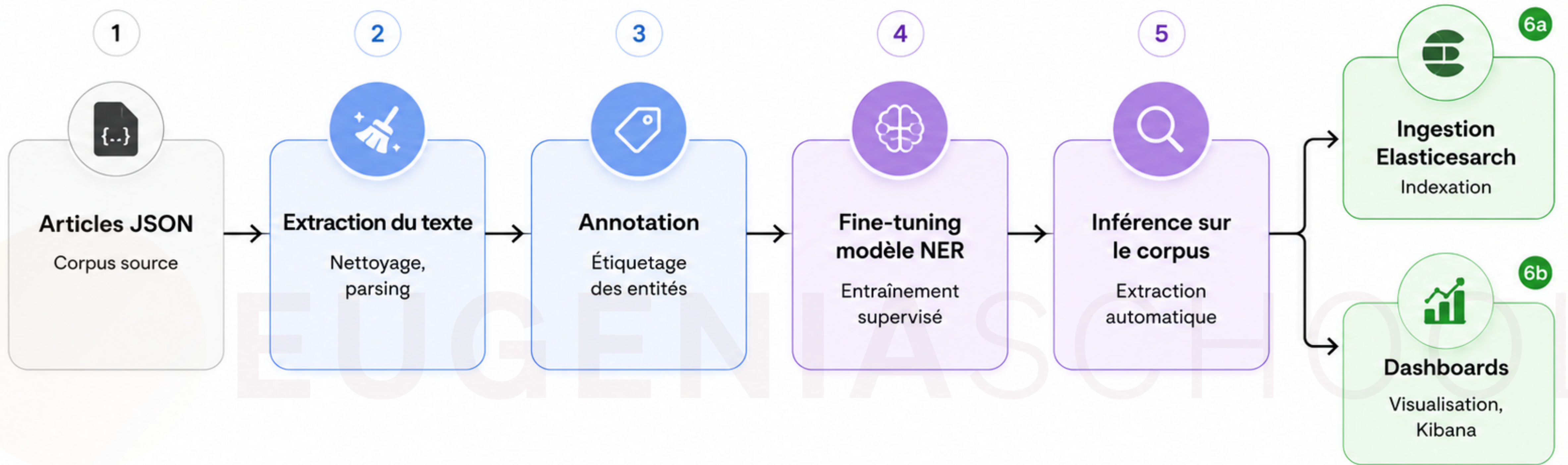
Chaque article contient un champ texte et plusieurs métadonnées (date, URL, titre, etc.).

L'objectif

Construire une chaîne de traitement complète qui part de texte brut et aboutit à des dashboards analytiques.

Le but : repérer si une tendance se dessine (quelles armes sont citées, quelles unités apparaissent, comment cela évolue dans le temps) et en tirer du renseignement.

Pipeline



Le cadre OSINT — à noter

Tass est un média d'État russe. Les données portent donc un **biais éditorial fort**.

On n'extrait jamais « la vérité ». On extrait ce qu'une source affirme.

La fiabilité et l'intention de la source font partie intégrante de l'analyse de renseignement. **Vous devrez en tenir compte dans l'interprétation finale.**

Pourquoi un NER et pas un LLM ?

Partie 2

Ce qu'est le NER

Named Entity Recognition :

Une tâche de classification token par token qui localise (offsets de début/fin) et catégorise (label) les entités d'un texte.

Exemple :

"The brigade fired Kalibr missiles near Kharkiv."

Entité	Label
brigade	MIL_UNIT
Kalibr	WEAPON
Kharkiv	LOC

Avantages du NER

- **Coût et vitesse** : *un petit modèle traite des milliers d'articles en local, sans appel API payant.*
- **Déterminisme** : *même entrée → même sortie. Crucial pour la traçabilité en renseignement.*
- **Sortie structurée native** : *positions exactes + labels, directement exploitables.*
- **Souveraineté** : *tout tourne en local, aucune donnée sensible envoyée à un tiers.*

Limites du NER

- Nécessite des **données annotées** pour apprendre un schéma personnalisé.
- Performances faibles sur le **contexte long**, la coréférence, les formulations ambiguës.
- Il ne « comprend » pas : il **reconnaît des motifs statistiques**.

Alors, à quoi sert le LLM ?

Le LLM est **trop coûteux, lent** et **non-déterministe** pour la production à l'échelle.

En revanche il est excellent pour **pré-annoter le corpus**.

EUGENIASCHOOL

Notre compromis : le LLM crée le dataset d'entraînement, le NER fait le travail de production.



Le modèle et le schéma

Partie 3

Pourquoi partir de **en_core_web_sm** ?

C'est le **petit modèle anglais** de spaCy : léger, rapide, adapté à une machine modeste. Il connaît déjà des entités génériques (PERSON, GPE, ORG...).

EUGENIASCHOOL

*On fait du **transfer learning** : on part d'un modèle déjà compétent plutôt que d'entraîner à zéro.*

Le problème du modèle générique

Le modèle de base sait repérer une organisation ou un lieu, mais il ne distingue pas :

- un **système d'armes** (ex. un missile)
- une **unité combattante** (ex. une brigade)
- une **organisation militaire** de haut niveau (ex. un état-major)

On doit lui apprendre notre propre schéma.

Notre schéma : 3 labels

Label	Definition	Exemple Typique
WEAPON	systèmes et matériels	missiles, blindés, drones, lance-roquettes
MIL_UNIT	unités combattantes	brigades, bataillons, régiments, divisions
MIL_ORG	organisations de haut niveau	états-majors, ministères de la Défense, forces armées

Pourquoi seulement 3 labels ?

- Assez **expressif** pour produire du renseignement utile.
- Assez **simple** pour être annoté et entraîné dans le temps d'un TP.

EUGENIASCHOOL

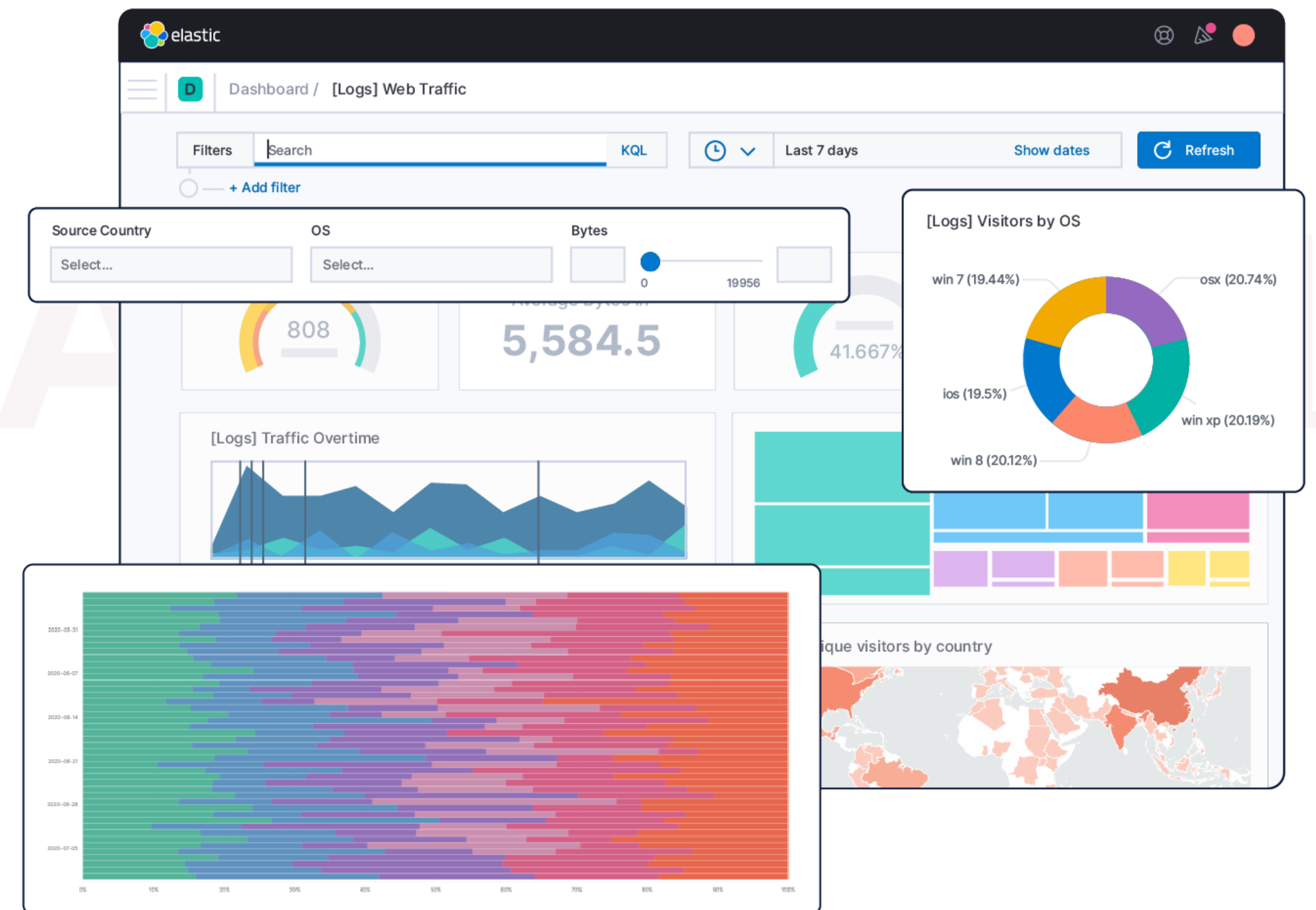
La cohérence de votre annotation compte plus que l'exhaustivité. Si les frontières sont ambiguës (ex. une garde nationale : UNIT ou ORG ?), choisissez une règle et appliquez-la partout.

Pourquoi Elasticsearch ?

Partie 4

Le besoin

Une fois les entités extraites, il faut pouvoir les **interroger** et les **visualiser**. Une pile de fichiers JSON ne permet pas l'analyse.



Ce qu'apporte un moteur de recherche

- **Recherche plein texte** rapide sur de gros volumes.
- **Agrégations** calculées côté serveur : top entités, séries temporelles, co-occurrences.
- **Dashboards** (Kibana / OpenSearch Dashboards) construits sans coder.



TP

Partie 5

EUGENIASCHOOL

Travail attendu

Un **document PDF respectant le template** unique contenant :

- Le lien GitHub vers votre code
- Des screenshots de vos dashboards
- Une analyse des métriques et de vos choix de dashboards
- Une note de renseignement courte

Étape 1 — Extraire le texte

Objectif : produire un corpus propre à partir de *articles.json*.

- Parser le JSON.
- Isoler le champ texte.
- Conserver le découpage par article, ne pas tout rassembler en un seul texte.
- Nettoyage minimal (espaces superflus, caractères parasites).

Étape 2a — Annoter le corpus

Objectif : produire un *annotations.json* au format spaCy.

Format spaCy attendu :

```
("The brigade fired Kalibr missiles.", {"entities": [(4, 11, "MIL_UNIT"), (18, 24, "WEAPON")]})
```

Étape 2b — Annoter le corpus

A. Les options

Label Studio, Doccano, Prodigy : interfaces d'annotation.

B. L'approche retenue : pré-annotation par LLM

On demande à un LLM de générer un premier jet d'annotations selon notre schéma.

Rédiger un prompt strict : imposer les 3 labels et un format de sortie structuré (offsets exacts).

C. Validation humaine — OBLIGATOIRE

- Relisez un échantillon d'annotations à la main.
- Vérifiez que les offsets correspondent bien au texte source.

Étape 3a — Ré-entraîner le modèle

✱ **Split train / dev**

Séparez vos données (ex. 80 % / 20 %) pour pouvoir évaluer honnêtement.

✱ **Conversion au format spaCy**

Convertissez vos annotations en fichiers binaires *.spacy* (via *DocBin*).

Étape 3b — Ré-entraîner le modèle

* Configuration

Générez une config spaCy partant de *en_core_web_sm* et déclarez vos nouveaux labels dans le pipe NER.

```
python -m spacy init config config.cfg --lang en --pipeline ner
```

Par défaut, la config crée un NER vierge. Pour réutiliser le modèle pré-entraîné comme point de départ, ajoutez dans *config.cfg*:

[components.ner]

source = "en_core_web_sm"

Étape 3c — Ré-entraîner le modèle

✱ Entraînement

```
python -m spacy train config.cfg \  
--output ./output \  
--paths.train ./train.spacy \  
--paths.dev ./dev.spacy
```

✱ Sauvegarde

Le meilleur modèle est sauvegardé dans `./output/model-best`.

Étape 4 — Inférer sur les articles

Objectif : appliquer votre modèle à une partie du corpus.

Que constatez-vous ?

Pour aller plus loin (optionnel)

GLiNER — l'inférence sans entraînement

Une alternative au modèle spaCy fine-tuné : extraire des entités décrites en langage naturel, sans aucun réentraînement.

	spaCy fine-tuné	GLiNER zero-shot
Entraînement	requis	aucun
Labels	figés	libres
Vitesse CPU	très rapide	plus lent

Étape 5 — Ingérer & visualiser

✱ **Deploiement du cluster**

Se rendre sur <https://cloud.elastic.co/> et se créer un compte avec 14 jours gratuits

✱ **Modéliser l'index**

Définissez un **mapping** : texte, date, entités structurées par type... en fonction de votre data.

```
{  
  "mappings": {  
    "properties": {  
      "text": { "type": "text" },  
      "date": { "type": "date" },  
      "weapons": { "type": "keyword" },  
      "mil_units": { "type": "keyword" },  
      "mil_orgs": { "type": "keyword" }  
    }  
  }  
}
```

Étape 5 — Ingérer & visualiser

* Ingérer

Transformez chaque article enrichi en document (regroupez les entités par label) et ingérez les.

* Construire les dashboards (exemples)

- **Top** WEAPON / MIL_UNIT / MIL_ORG (agrégation terms).
- **Histogramme temporel** des mentions (date_histogram).
- **Co-occurrences** d'entités.
- **Filtres** par période.